Kubron: план разработки виджетной платформы

Версия 0.1 · 27.10.2026 · черновик для тимлида

1. Цель

Overview превращается в рабочее поле виджетов. Требования владельца продукта:

- каждая плашка — отдельный виджет: разрабатывается отдельно, подключается в Kubron;
- виджет любого размера, перетаскивается в любое место поля;
- положение сохраняется отдельно для каждого кластера (и пользователя); плашки можно скрывать;
- виджеты сторонних авторов и маркетплейс по аналогии с расширениями VS Code;
- работает и в десктопе (Wails), и в веб-режиме.

Связанный документ: «Kubron: требования виджетной платформы к веб-версии» (что мы просим у команды веба).

2. Ключевые решения (предложения для утверждения)

1. **Один контракт, два рантайма.** Встроенные виджеты — доверенные Vue-компоненты; сторонние — в iframe-песочнице. Манифест, размеры, конфиг и SDK общие.
2. **Манифест — сериализуемый JSON** без ссылок на код приложения.
3. **Виджет не вызывает `callApp`.** Он говорит с хостом через SDK (`postMessage`). Данные запрашивает и раздаёт хост (Data Broker) с дедупликацией запросов.
4. **Разрешения (capabilities):** виджет заявляет, пользователь или админ подтверждает, хост проверяет каждый вызов. Реальные права — пересечение разрешений виджета и прав пользователя (`can()`; источник истины — Go).
5. **Сторонние виджеты v1 — только чтение.** Запись — отдельное разрешение позже, с подтверждением от хоста.
6. **Сеть виджета — только через хост-прокси** с allowlist доменов; токены и секреты виджет не видит.

7. **Раскладка:** сетка 12 колонок, элемент `{id, type, x, y, w, h, config}`, хранение на пару «пользователь + кластер».
8. **Пакеты подписаны**, есть список отзыва (kill-switch).
9. **SDK — самый дорогой в изменении публичный API:** `apiVersion`, поддержка N и N-1, ранний пилот.

3. Не входит в первую версию

- платные виджеты, рейтинги, отзывы;
- виджеты вне Overview (контракт этому не мешает);
- отдельная раскладка для мобильных (узкое окно — автоматическая колонка);
- запись из сторонних виджетов.

4. Архитектура

Компонент	Слой	Назначение
Манифест	контракт	id, версия, `apiVersion`, размеры, разрешения, схема настроек
Реестр типов	frontend	встроенные через `import.meta.glob`, установленные — через сервер
Widget Host	frontend	`NativeHost` и `SandboxHost`: жизненный цикл, ошибки, размеры, тема
Data Broker	frontend (+ Go)	общий слой данных: подписки, дедупликация, опрос только видимых виджетов
Permission Broker	frontend + Go	проверка разрешений и прав пользователя на каждый вызов
Layout Engine	frontend	сетка, перетаскивание, ресайз, коллизии, режим редактирования, клавиатура
Layout Store	frontend → Go	сначала `localStorage`, затем серверные UI-настройки (R4 из документа требований)
Пакеты и реестр	Go + сервис	формат, подпись, установка, обновления, отзыв

Компонент	Слой	Назначение
`@kubron/widget-sdk`	npm	типы, клиент `postMessage`, токены темы, тестовый стенд
Маркетплейс	сервис + UI	каталог, публикация, модерация

5. Манифест (черновик)

```
{
  "id": "acme.cert-expiry",
  "name": "Cert expiry",
  "version": "1.2.0",
  "apiVersion": 1,
  "publisher": "acme",
  "entry": "index.html",
  "size": { "default": [4, 4], "min": [3, 3], "max": [12, 8] },
  "permissions": ["data:insights.certs"],
  "requires": ["secrets.read"],
  "config": { "schema": "config.schema.json" }
}
```

- Размеры — в ячейках сетки. У встроенных виджетов вместо `entry` указан `component`.
- `permissions` — закрытый перечень; `secrets.read` сторонним виджетам не выдаётся.
- `requires` — права пользователя, без которых виджет не показывается (как у «Expiring certs» сегодня).
- `config.schema` — JSON Schema; форму настроек рисует хост, поэтому виджету не нужен свой UI настроек.

6. Раскладка и хранение

- **Сетка:** 12 колонок, фиксированная высота ряда (предварительно 80 px, уточнить на прототипе), отступ 14 px, как сейчас.
- **Данные:** `{ "v": 1, "items": [{ "id": "w-1", "type": "builtin.workloads", "x": 0, "y": 4, "w": 8, "h": 6, "hidden": false, "config": {} }] }`.
- **Ключ хранения:** `kubron.overviewLayout/<userId>/<clusterId>`, запись версионирована; интерфейс хранилища заменяем на серверный (R4).

- **Нормализация при загрузке:** неизвестный тип остаётся заглушкой «Widget not installed» (позиция не теряется); новые встроенные виджеты ставятся в первое свободное место, а не поверх; значения за границами сетки обрезаются.
- **Узкое окно:** порядок по (y, x) в одну колонку; сохранённая раскладка не меняется.
- **Режим редактирования:** кнопки Customize, Done, Reset; перемещение и изменение размера мышью и клавиатурой, `aria`-подписи; скрытые виджеты — в панели «Скрытые».
- **Движок:** кандидаты — `gridstack`, `grid-layout-plus` и своя реализация на CSS grid и pointer events. Критерии: Vue 3, клавиатура и a11y, коллизии и ресайз, размер, поддержка, лицензия, поведение в WebView2, WKWebView и WebKitGTK. Эти библиотеки мы глубоко не проверяли, выбор — по результатам прототипа (2–3 дня) в фазе 1.
- **Экономия:** данные скрытого или невидимого виджета не опрашиваются.

7. Безопасность (сводка)

- **Песочница:** `<iframe sandbox="allow-scripts">` без `allow-same-origin`; содержимое отдаёт наш обработчик со строгим CSP (`connect-src 'none'`).
- **Канал:** только `postMessage` с версионированным протоколом; хост проверяет `event.source` и схему каждого сообщения.
- **Лимиты:** размер пакета, число сторонних виджетов на экране, частота запросов, сторожевой таймер с отключением зависших.
- **Секреты:** `secrets.read` сторонним не выдаётся; kubeconfig и токены недоступны.
- **Аудит:** действия виджета помечаются полем `via`.
- **Встроенные виджеты** работают в основном окне и проходят обычное ревью кода.
- Подробности и требования к веб-версии — в отдельном документе.

8. Встроенные виджеты

Текущие карточки Overview становятся встроенными виджетами. Источники данных переходят в Data Broker; клики, которые сейчас вызывают навигацию, идут через хост (`host.navigate`, `host.openResource`).

Виджет	Размер по умолчанию (колонки)	Источник данных	Заметки
`stat-nodes`, `stat-pods`, `stat-restarts`, `stat-warnings`	3	`overview`	четыре плашки сверху; клик ведёт в раздел
`resource-usage`	4	`metrics.cluster`, `overview.requests`	сейчас сам опрашивает метрики; действие «открыть настройки»
`top-pods`, `storage`	4	`hotspots`	
`workloads`	8	`overview.workloads`	открытие ресурса
`problem-pods`, `warnings`	4	`overview`	сейчас стоят столбцом рядом с Workloads
`recent-changes`	4	`insights.changes`	окно времени (1h/6h/24h) переходит в конфиг экземпляра
`capacity`, `blockers`	4	`insights.risks`	
`expiring-certs`	4	`insights.certs`	`requires: secrets.read`
`helm`	4	`helm.releases`	сейчас сам загружает данные; скрыт, если релизов нет

Раскладка по умолчанию может повторить сегодняшний вид почти один в один: свободные координаты позволяют поставить `warnings` под `problem-pods` справа от `workloads`. Исключение — нижний ряд, который сейчас адаптивно

подстраивается под ширину (auto-fit), а станет трёх-колоночным. Компонент `ClusterHealthCard` есть в кодовой базе, но по нашему поиску к Overview не подключён — кандидат в виджеты.

9. Фазы

Оценки грубые и рассчитаны на одного фронтенд-разработчика с поддержкой по Go; уточнить после фазы 0.

Фаза 0. ADR и модель угроз (≈ 1 неделя)

- ADR «Виджетная платформа»: контракт, два рантайма, модель угроз, манифест v0, решение по песочнице.
- Передать документ требований команде веба и получить ответы на вопросы.
- **Выход:** ADR утверждён тимлидом.

Фаза 1. Нативные виджеты и раскладка (≈ 3–5 недель, от веба не зависит)

1. Прототип выбора движка раскладки (2–3 дня); проверка в трёх WebView и в браузере.
2. Манифест v0 и реестр встроенных виджетов.
3. Data Broker: перенос `useOverview`, `useHotspots`, `useInsights`; опрос только видимых виджетов.
4. `NativeHost`: error boundary, состояния loading и error, передача размера и контекста (кластер, тема, навигация).
5. Layout Engine, режим редактирования, клавиатура и a11y, скрытие и сброс.
6. Layout Store: интерфейс и `localStorage`, нормализация, версия схемы.
7. Раскладка по умолчанию, повторяющая текущий вид; визуальная проверка.

Выход: пользователь перемещает, меняет размер и скрывает виджеты; раскладка своя для кластера и переживает перезапуск; работает в десктопе и вебе; тесты зелёные.

Фаза 2. SDK и песочница (≈ 5–8 недель; после этапа 2 веба и утверждения ADR-0004 и ADR-0012)

1. Протокол `postMessage` v1: handshake, размеры, тема, `query` и `subscribe`, ошибки, версия.
2. `SandboxHost`: iframe, CSP, лимиты, сторожевой таймер.
3. Permission Broker и диалог разрешений.
4. Форма настроек по JSON Schema.

5. `@kubron/widget-sdk`, шаблон проекта, режим разработчика (адрес на loopback, с предупреждением).
6. Локальная установка из папки или файла (десктоп).
7. Набор «враждебных» тестовых виджетов и security-тесты.

Выход: пилот из 2–3 сторонних виджетов, написанных не авторами платформы; попытки выйти из песочницы блокируются.

Фаза 3. Пакеты и реестр (≈ 4–6 недель)

1. Формат пакета (`.kwidget`: манифест, бандл, подпись), семвер.
2. Подпись Ed25519, издатели, хэши в lock-файле.
3. Установка и обновление с диалогом разрешений.
4. Список отзыва; клиент проверяет его периодически.
5. Веб: серверная установка администратором, политики `--widgets-*`, аудит.
6. Приватные реестры и офлайн-режим для изолированных контуров.

Фаза 4. Маркетплейс (≈ 4–8+ недель, зависит от оргмодели)

1. Каталог: сначала статический индекс, затем сервис.
2. Публикация через CLI и автоматические проверки (манифест, размер, запрещённые API).
3. Модерация, жалобы, процесс отзыва.
4. Каталог в интерфейсе приложения.
5. Условия использования и лицензии — согласовать с ADR-0013 и юристом до публичного запуска.
6. Старт с курируемого каталога, открытая публикация позже.

Фаза 5. Развитие

Запись из виджетов (с подтверждением), декларативные виджеты без кода, виджеты на других экранах, платные виджеты.

10. Тестирование

- **Юнит (vitest, окружение node, без DOM):** нормализация раскладки, коллизии, миграции схемы, ключи хранения, разбор манифеста, проверка разрешений.
- **E2E (Playwright, уже есть в проекте):** перетаскивание и ресайз, сохранение после перезагрузки, сброс, раскладка своя для кластера.

- **Безопасность:** «враждебные» виджеты (доступ к родителю, `fetch` к `/api`, `localStorage`, навигация `window.top`, всплывающие окна, отправка форм, зависание), проверка CSP и Origin, fuzz протокола `postMessage`.
- **Доступность:** полный сценарий только с клавиатуры.
- **Совместимость:** ручной чеклист на Windows (WebView2), macOS (WKWebView), Linux (WebKitGTK) и в браузерах веб-режима: перетаскивание, pointer events, sandbox.

11. Риски

Риск	Мера
Веб-ветка меняет основание (сессии, хранилище)	делить работу по фазам; фаза 1 от веба не зависит; хранилища за интерфейсами
Различия WebView (перетаскивание, CSP, sandbox)	движок на pointer events, ручная проверка на трёх ОС уже в фазе 1
Память и время загрузки iframe	лимит числа сторонних виджетов, ленивая загрузка, декларативные виджеты в фазе 5
SDK придётся ломать после выхода	ранний пилот на 2–3 виджетах, `apiVersion`, поддержка N и N-1
Вредоносные виджеты в каталоге	подпись, ревью, отзыв, только чтение в v1, ограничение разрешений
Нагрузка модерации и поддержки маркетплейса	курируемый каталог на старте, открытая публикация позже
Лицензионные и правовые вопросы	показать условия юристу до публичного запуска (см. ADR-0013)
Лишняя нагрузка на бэкенд кластера	Data Broker с дедупликацией, опрос только видимого, лимиты частоты

12. Открытые вопросы к тимлиду

1. Утвердить подход «один контракт, два рантайма».
2. Область установки в вебе: организация или команда (связано с ADR-0012)?
3. Каталог курируемый или открытый на старте?

4. Нужна ли декларативная форма виджета уже в SDK v1?
5. Кто владеет инфраструктурой реестра: хостинг, ключи подписи, модерация?
6. Нужны ли платные виджеты (влияет на ADR-0013)?
7. Минимальные поддерживаемые версии WebView на десктопе.

13. Ближайшие шаги

1. Передать документ «требования к веб-версии» команде веба.
2. Подготовить ADR «Виджетная платформа».
3. Фаза 1: прототип выбора движка раскладки (2–3 дня).